

Structured Pruning of Large Language Models with LLM-Pruner

Zarek Lu
Rice University
Houston, Texas, USA
zfl1@rice.edu

Abstract

This project studies LLM-Pruner, a structured pruning framework for large language models. The goal of the project is to reproduce the pruning stage of the system, evaluate its behavior under practical hardware constraints, and investigate whether changing the calibration dataset affects prune-only model quality. I ran prune-only experiments on a LLaMA-7B-compatible checkpoint using block-wise Taylor pruning, compared multiple calibration datasets, and analyzed the resulting tradeoffs in perplexity, memory usage, and parameter retention. The results show that calibration dataset choice affects post-prune quality even when the pruning ratio and resulting model size remain fixed, highlighting calibration data as an important design choice for structured pruning systems.

1 Introduction

LLM-Pruner addresses the problem of reducing the size and inference cost of large language models without rebuilding them from scratch. This is important because modern LLMs often require too much memory and computation for efficient deployment. It provides an optimization and pruning framework: given a pre-trained transformer, it removes groups of parameters such as attention and MLP components. Its key contribution is a task-agnostic, training-free pruning pipeline that combines dependency-aware structured pruning with importance estimation methods such as Taylor-based scoring, and then optionally supports post-pruning recovery through lightweight fine-tuning. This project is relevant to deep learning system design because it sits at the intersection of model architecture, memory efficiency, and deployability: it shows how system-level constraints like GPU memory, parameter count, and inference latency can be improved through algorithmic changes to model structure, making it a useful example of how optimization techniques can turn large pretrained models into more practical systems.

2 Code Setup and Environment

I set up LLM-Pruner first on a local Windows machine and later attempted to reproduce the workflow on a Linux virtual machine/cluster environment. The local machine used an NVIDIA GPU with 16 GB of VRAM. This was enough, as the pruning computation itself was kept on CPU and only evaluation was moved to CUDA. The software environment required several version adjustments for compatibility. In particular, the project did not work reliably with very new libraries, so I used Python 3.11/3.9-compatible environments, PyTorch with CUDA support, Transformers 4.31.0, Tokenizers 0.13.3, Datasets 2.14.7, and SentencePiece. Installation involved creating a virtual environment, installing CUDA-enabled PyTorch, and then pinning the Hugging Face libraries to older versions that matched the expectations of the repository. During setup, I encountered multiple issues: Windows shell scripts had

line-ending problems, the default model identifier in the repository pointed to an inaccessible older LLaMA checkpoint, newer versions of transformers, datasets, and torch introduced breaking changes, and CPU-only PyTorch caused CUDA evaluation errors. These were resolved by converting scripts to Unix-style line endings when needed, replacing the default base model with `baffo32/decapoda-research-llama-7B-hf`, downgrading incompatible Hugging Face packages, patching `post_training.py` for newer torch.load behavior, and installing a CUDA-enabled PyTorch build.

3 Evaluation Platform

The main evaluation was ultimately performed on my local machine, since it was the only platform on which I could successfully complete the pruning stage end-to-end. The local system used one NVIDIA GPU (16 GB VRAM) together with CPU execution for the pruning importance computation and GPU execution for evaluation. I also attempted to run the system on NOTS, but that platform imposed strict disk quota limits, which prevented reliable installation of the full CUDA software stack and large Hugging Face model downloads. For this reason, the local machine was the more suitable platform for evaluating the artifact.

4 Implementation and Deployment

I used the repository's main pruning entry point, `hf_prune.py`, as the core script for the experiment. The pruning runs were executed with `baffo32/decapoda-research-llama-7B-hf` using block-wise Taylor pruning with a pruning ratio of 0.25, attention and MLP pruning applied to middle transformer layers, and evaluation enabled after pruning. To support my experiment, I modified the code in two ways. First, I extended the calibration data pipeline so that the pruning script could use multiple calibration datasets instead of only the default BookCorpus, allowing comparison against WikiText-2 and PTB. Second, I refactored `hf_prune.py` so it could sweep over multiple calibration datasets and print the resulting pruning metrics in a summary table. I also made compatibility edits in `post_training.py`, although I ultimately did not run the recovery/post-training stage because it was too computationally expensive for the available setup; in practice, it would have taken more than 30 hours to complete. Therefore, only the pruning performance was evaluated. No containerization or distributed launch was used, since execution was done directly from the command line in a single-process environment.

5 Results

I evaluated LLM-Pruner in a prune-only setting using four calibration dataset choices: the default BookCorpus, plus WikiText-2, PTB, and C4. All successful runs used the same base model, pruning ratio, pruning method, and layer ranges; only the calibration dataset was

Table 1: Prune-only results under different calibration datasets.

Dataset	Params After	Retain %	W2 PPL	PTB PPL	Mem. (MiB)
BookCorpus	5,422,977,024	80.48	19.09	34.23	10383.65
WikiText-2	5,422,977,024	80.48	18.50	35.11	10383.65
PTB	5,422,977,024	80.48	20.24	32.41	10383.65
C4	—	—	—	—	—

changed. Across the three successful runs, the pruning statistics were identical: the model was reduced from 6,738,415,616 parameters to 5,422,977,024 parameters, retaining 80.48% of the original parameters after pruning. The estimated memory requirement after pruning was also unchanged at 10,383.65 MiB, indicating that the calibration dataset primarily affected model quality rather than the structural size of the pruned network.

In terms of quality, the calibration dataset had a noticeable effect on post-prune perplexity. Using the default BookCorpus calibration data produced a WikiText-2 perplexity of 19.09 and PTB perplexity of 34.23. Replacing BookCorpus with WikiText-2 improved WikiText-2 perplexity to 18.50, but worsened PTB perplexity to 35.11. In contrast, using PTB as the calibration dataset worsened WikiText-2 perplexity to 20.24, but improved PTB perplexity to 32.41. This suggests that the calibration dataset influences which linguistic distribution is better preserved after pruning. The C4 experiment did not complete successfully, so no results were obtained for that dataset. These results are summarized in Table 1.

Relative to the project baseline, BookCorpus serves as the default calibration dataset in the repository, so the WikiText-2 and PTB runs can be interpreted as direct comparisons against that baseline. Compared with BookCorpus, WikiText-2 calibration improved performance on WikiText-2 but degraded PTB, while PTB calibration improved PTB but degraded WikiText-2. This indicates that neither alternative was uniformly better than the baseline across both metrics. Compared with the authors' reported results, my BookCorpus prune-only numbers closely matched the prune-only behavior reported in the original paper and repository, especially the post-prune perplexity scale for LLaMA-7B-style pruning, which indicates that my pruning-stage reproduction was broadly successful. The paper reports stronger final results after the recovery stage, but since I did not run post-training, my results should be compared only to the prune-only stage rather than the paper's recovered model. LLM-Pruner reports that even with around 20% pruning, the compressed model can maintain most of the original model's performance, and the paper's central contribution is that dependency-aware pruning plus lightweight recovery substantially outperforms naive pruning baselines.

6 Observations

The main observation from these experiments is that the calibration dataset matters, even when all other pruning settings are held fixed. Because the parameter count, retention ratio, and memory footprint were identical across runs, the only meaningful difference came from how the calibration samples shaped the Taylor-based importance estimates during pruning. The results suggest that the

pruning process preserves information most effectively for distributions that resemble the calibration dataset. For example, calibrating on WikiText-2 led to the best WikiText-2 perplexity, while calibrating on PTB led to the best PTB perplexity. This pattern suggests a form of dataset-specific bias in the pruning importance estimates: the model retains structures that are most useful for the type of text it sees during calibration.

Another observation is that resource constraints strongly shaped the feasible evaluation workflow. On the local machine, the pruning stage itself was achievable, but full post-training was too computationally expensive to complete in a reasonable amount of time. On the Linux VM / cluster environment, disk quota limits and large model downloads made it difficult even to maintain a complete CUDA-enabled setup for repeated experiments. This reflects an important systems property of LLM-Pruner: while the pruning algorithm is designed to reduce model size and inference cost, the pruning process itself is still computationally demanding, especially for large LLaMA-family checkpoints. In practice, the configuration of CPU for pruning computation and CUDA for evaluation was the most stable compromise for available hardware.

A final observation is that the system is sensitive to software versions and environment configuration. I encountered multiple compatibility issues related to Python, PyTorch, Hugging Face Transformers, and Datasets. This sensitivity likely arises because the repository was developed against an older software stack, while current libraries have changed APIs and loading behavior. As a result, reproducing the pruning workflow required not only running the published scripts, but also adapting the environment carefully to match the assumptions of the original code. This highlights a common issue in deep learning systems work: artifact reproducibility often depends as much on environment management as on the algorithm itself.

7 Improvements and Extensions

The main improvement I implemented was an extension of the pruning pipeline to support multiple calibration datasets rather than only the default BookCorpus. I modified the dataset loading logic so that pruning could be run with BookCorpus, WikiText-2, PTB, and C4, and I refactored the pruning script to sweep across calibration datasets and summarize the resulting metrics in a table. This change made it possible to evaluate how calibration data affects prune-only quality under otherwise fixed settings. The impact of this modification was clear: different calibration datasets produced different perplexity tradeoffs, confirming that calibration data is a meaningful design choice in structured pruning.

I also attempted, but did not fully complete, several additional optimizations. One was running the system on a Linux VM / cluster environment to accelerate experimentation, but this was limited by disk quota constraints and large model dependencies. Another was completing the C4 calibration experiment, which ultimately failed and therefore remains unfinished. I also explored the recovery/post-training stage, but did not complete it because the computational cost was too high for the available hardware and time budget. These unfinished efforts still helped clarify the system's bottlenecks: storage requirements, environment fragility, and the high runtime cost of post-pruning recovery.

For future work, the most promising extension would be to expand the calibration-data study into a more systematic experiment. One direction would be to test whether mixed calibration datasets or task-aligned calibration sets perform better than a single corpus. Another would be to evaluate whether the same calibration trends hold at different pruning ratios, since it is possible that the best calibration dataset depends on pruning aggressiveness. A second useful

extension would be to complete the recovery stage on a more capable GPU platform and compare whether the differences observed in the prune-only stage persist after post-training. More broadly, these results suggest that calibration dataset selection is a practical knob for improving structured pruning quality, and therefore a worthwhile target for future optimization in LLM-Pruner.